

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

Guilherme de Oliveira Lacerda

SISTEMA DE VENDAS ALIMENTÍCIAS

CAMPOS DO JORDÃO

2026

Guilherme de Oliveira Lacerda

Sistema de Vendas Alimentícias

Projeto final da matéria Banco de Dados II
em Análise e Desenvolvimento de Sistemas
pelo Instituto Federal de São Paulo - IFSP.

Orientador: Prof. Paulo Giovani de Faria
Zeferino

CAMPOS DO JORDÃO

2026

RESUMO

Este trabalho apresenta o desenvolvimento de um banco de dados relacional projetado para otimizar a gestão do ciclo completo de vendas em uma empresa. O objetivo principal consiste em estruturar uma base de dados segura e eficiente, capaz de gerenciar desde o pedido até a logística e faturamento, diminuindo erros e possibilitando relatórios gerenciais complexos. A metodologia adota a abordagem clássica de ciclo de vida de banco de dados, abrangendo a análise de requisitos e o mapeamento das regras de negócio. Como aporte teórico, conceitos de projeto e administração de sistemas de bancos de dados de Peter Rob e Carlos Coronel, associados à exploração prática da linguagem estruturada de consultas proposta por Thomas Nield. O desenvolvimento utiliza o software brModelo para a criação dos modelos conceitual e lógico, enquanto a implementação do modelo físico ocorre no ambiente PostgreSQL (DB Fiddle) através de uma arquitetura com nove tabelas interligadas: Item, Pedido, Vendedor, Cliente, Romaneio, Transportadora, Pagamento, Nota Fiscal e Produto. O projeto valida o sistema com a aplicação de trinta consultas SQL avançadas, nas quais cada comando engloba junções, agrupamentos e subconsultas para a geração de relatórios gerenciais. O estudo conclui que a modelagem proposta atende com rigor aos requisitos operacionais da cadeia de vendas e promove a consistência informacional.

Palavras-Chave: Banco de Dados Relacional; PostgreSQL; Sistema de Vendas; Modelagem de Dados; Rob & Coronel.

LISTA DE ILUSTRAÇÕES

FIGURA 1 - Modelo Conceitual (brModelo)	13
FIGURA 2 - Modelo Lógico (brModelo)	14
FIGURA 3 - Modelo Físico (brModelo)	15
FIGURA 4 - Modelo Físico (brModelo)	16
FIGURA 5 - Modelo Físico (brModelo)	17
FIGURA 6 - Consulta 1 (PostgreSQL)	23
FIGURA 7 - Consulta 2 (PostgreSQL)	24
FIGURA 8 - Consulta 3 (PostgreSQL)	24
FIGURA 9 - Consulta 4 (PostgreSQL)	25
FIGURA 10 - Consulta 5 (PostgreSQL)	25
FIGURA 11 - Consulta 6 (PostgreSQL)	26
FIGURA 12 - Consulta 7 (PostgreSQL)	26
FIGURA 13 - Consulta 8 (PostgreSQL)	27
FIGURA 14 - Consulta 9 (PostgreSQL)	27
FIGURA 15 - Consulta 10 (PostgreSQL)	28
FIGURA 16 - Consulta 11 (PostgreSQL)	28
FIGURA 17 - Consulta 12 (PostgreSQL)	29
FIGURA 18 - Consulta 13 (PostgreSQL)	29
FIGURA 19 - Consulta 14 (PostgreSQL)	30
FIGURA 20 - Consulta 15 (PostgreSQL)	30
FIGURA 21 - Consulta 16 (PostgreSQL)	31
FIGURA 22 - Consulta 17 (PostgreSQL)	31
FIGURA 23 - Consulta 18 (PostgreSQL)	32

FIGURA 24 - Consulta 19 (PostgreSQL)	32
FIGURA 25 - Consulta 20 (PostgreSQL)	33
FIGURA 26 - Consulta 21 (PostgreSQL)	33
FIGURA 27 - Consulta 22 (PostgreSQL)	34
FIGURA 28 - Consulta 23 (PostgreSQL)	34
FIGURA 29 - Consulta 24 (PostgreSQL)	35
FIGURA 30 - Consulta 25 (PostgreSQL)	35
FIGURA 31 - Consulta 26 (PostgreSQL)	36
FIGURA 32 - Consulta 27 (PostgreSQL)	36
FIGURA 33 - Consulta 28 (PostgreSQL)	37
FIGURA 34 - Consulta 29 (PostgreSQL)	37
FIGURA 35 - Consulta 30 (PostgreSQL)	38

LISTA DE ALGORITMOS

ALGORITMO 1 - Criação das Tabelas (PostgreSQL)	17
ALGORITMO 2 - Inserção de Dados (PostgreSQL)	19
ALGORITMO 3 - Consulta 1 (PostgreSQL)	23
ALGORITMO 4 - Consulta 2 (PostgreSQL)	24
ALGORITMO 5 - Consulta 3 (PostgreSQL)	24
ALGORITMO 6 - Consulta 4 (PostgreSQL)	25
ALGORITMO 7 - Consulta 5 (PostgreSQL)	25

ALGORITMO 8 - Consulta 6 (PostgreSQL)	26
ALGORITMO 9 - Consulta 7 (PostgreSQL)	26
ALGORITMO 10 - Consulta 8 (PostgreSQL)	27
ALGORITMO 11 - Consulta 9 (PostgreSQL)	27
ALGORITMO 12 - Consulta 10 (PostgreSQL)	28
ALGORITMO 13 - Consulta 11 (PostgreSQL)	28
ALGORITMO 14 - Consulta 12 (PostgreSQL)	29
ALGORITMO 15 - Consulta 13 (PostgreSQL)	29
ALGORITMO 16 - Consulta 14 (PostgreSQL)	30
ALGORITMO 17 - Consulta 15 (PostgreSQL)	30
ALGORITMO 18 - Consulta 16 (PostgreSQL)	31
ALGORITMO 19 - Consulta 17 (PostgreSQL)	31
ALGORITMO 20 - Consulta 18 (PostgreSQL)	32
ALGORITMO 21 - Consulta 19 (PostgreSQL)	32
ALGORITMO 22 - Consulta 20 (PostgreSQL)	33
ALGORITMO 23 - Consulta 21 (PostgreSQL)	33
ALGORITMO 24 - Consulta 22 (PostgreSQL)	34
ALGORITMO 25 - Consulta 23 (PostgreSQL)	34
ALGORITMO 26 - Consulta 24 (PostgreSQL)	35
ALGORITMO 27 - Consulta 25 (PostgreSQL)	35
ALGORITMO 28 - Consulta 26 (PostgreSQL)	36
ALGORITMO 29 - Consulta 27 (PostgreSQL)	36
ALGORITMO 30 - Consulta 28 (PostgreSQL)	37
ALGORITMO 31 - Consulta 29 (PostgreSQL)	37
ALGORITMO 32 - Consulta 30 (PostgreSQL)	38

SUMÁRIO

1. INTRODUÇÃO	09
1.1 Objetivos	09
1.2 Justificativa	09
1.3 Aspectos Metodológicos	10
1.4 Aporte Teórico	10
2. FUNDAMENTAÇÃO TEÓRICA	11
2.1 Análise de Requisitos e Regras de Negócio	11
2.2 Regras de Negócio do Sistema de Vendas	12
2.3 Trabalhos Relacionados	12
3. SISTEMA DE VENDAS	13
3.1 Abstração de Dados	13
3.2 Criação de Tabelas e Inserção de Dados (PostgreSQL)	17
3.3 Consultas Propostas	22
4. CONCLUSÃO	42
REFERÊNCIAS	43

1. INTRODUÇÃO

Este trabalho da disciplina de Banco de Dados II se concentra em criar um sistema de banco de dados relacional completo para uma empresa de vendas. A ideia principal é desenvolver uma base de dados que consiga gerenciar todo o ciclo de uma venda, desde o pedido feito pelo cliente até a emissão da nota fiscal e a logística de entrega. O objetivo é ter uma base de dados bem estruturada para evitar erros e conseguir gerar relatórios complexos sobre o desempenho de vendas, clientes e produtos usando comandos SQL avançados.

1.1 Objetivos

O objetivo geral do projeto é planejar, modelar e construir um banco de dados relacional que gerencie com segurança todo o processo de vendas de uma empresa, garantindo que as informações estejam prontas para relatórios gerenciais complexos usando SQL.

Como objetivos específicos, temos:

- Estudar como funciona o processo de vendas (pedido, faturamento, expedição) para definir as regras de negócio de como a informação vai ser guardada no sistema;
- Criar os modelos conceitual e lógico do banco de dados usando o Diagrama Entidade-Relacionamento, com o auxílio da ferramenta BrModelo;
- Implementar o modelo físico no SGBD PostgreSQL, com uma estrutura de nove tabelas interligadas: Item, Pedido, Vendedor, Cliente, Romaneio, Transportadora, Pagamento, Nota Fiscal e Produto;
- Verificar se o modelo está consistente e funcionando bem com a criação e execução de 30 consultas SQL complexas, simulando relatórios de gestão.

1.2 Justificativa

Este projeto é importante por dois motivos. Primeiro, academicamente, ele serve para colocar em prática tudo o que aprendemos sobre modelagem e manipulação de dados na matéria de Banco de Dados II. Segundo, na prática, ele atende a uma demanda real das empresas por um sistema centralizado de vendas

que ajude a controlar melhor o estoque, o fluxo de pedidos, os pagamentos e o desempenho individual dos vendedores. Ter essa base de dados organizada é crucial para tomar decisões mais rápidas e com menos chance de erro.

1.3 Aspectos Metodológicos

A metodologia adotada combina a pesquisa bibliográfica em literaturas especializadas (referências de Rob, Coronel e Nield) com a fase de execução prática do desenvolvimento. Para as etapas de design (modelos conceitual e lógico), utiliza-se o software BrModelo. A implementação física do banco de dados e a fase de testes são realizadas no ambiente PostgreSQL (DB Fiddle), que é onde o código será rodado. O processo de desenvolvimento compreende: o levantamento do fluxo de vendas, o mapeamento relacional das nove tabelas, a inclusão de dados de teste e, por fim, a execução das trinta consultas SQL para confirmar a capacidade do sistema em gerar os relatórios necessários.

1.4 Aporte Teórico

O aporte teórico que fundamenta este trabalho é baseado nas obras de Peter Rob e Carlos Coronel, que fornecem os princípios essenciais do modelo relacional e as técnicas de normalização para manter a integridade dos dados. Adicionalmente, utiliza-se o material de Thomas Nield para aprofundar as habilidades na linguagem SQL, aspecto vital para a construção das consultas avançadas destinadas à validação do projeto.

2 FUNDAMENTAÇÃO TEÓRICA

O embasamento teórico para este projeto está centrado no Modelo Relacional, visto que a estrutura de tabelas é a base para a organização dos dados de vendas. Foca-se nas regras de normalização de dados para garantir que não ocorra redundância e inconsistência nas informações. Também, é explorada a aplicação avançada da linguagem SQL no ambiente do SGBD, o PostgreSQL, para permitir a geração de relatórios complexos.

Análise de Requisitos e Regras de Negócio

A modelagem do banco de dados seguiu a abordagem de níveis de abstração, conforme sugerido pela literatura clássica (Rob e Coronel). Essa divisão permite estruturar o projeto desde a fase de entendimento do negócio até a implementação real:

- **Modelo Conceitual:** Representa a visão de alto nível do sistema de vendas, identificando entidades como Pedido, Produto e Cliente, e os relacionamentos entre elas. É a base do projeto, criada sem depender de qual SGBD será usado, com o auxílio do software BrModelo.
- **Modelo Lógico:** Traduz o conceitual para a estrutura do Modelo Relacional, definindo as nove tabelas e suas chaves primárias e estrangeiras. Nessa fase, as regras de negócio são transformadas em restrições de integridade, como as FOREIGN KEY definidas no código SQL.
- **Modelo Físico:** É a implementação prática do código SQL no SGBD. Neste projeto, a execução dos scripts de criação de tabelas e de inserts de dados é realizada no ambiente PostgreSQL (DB Fiddle), onde os tipos de dados e os comandos de restrição de integridade são definidos.

2.2. Regras de Negócio do Sistema de Vendas

Para que o banco de dados funcionasse corretamente, foi preciso definir as regras de negócio do processo de vendas. Essas regras garantem que o sistema reflita a realidade de uma operação comercial e que a integridade dos dados seja mantida nas nove tabelas:

- **RN01 – Estrutura do Pedido:** Um Pedido é composto por um ou mais Itens. Um item deve estar ligado a um único Pedido e a um único Produto.
- **RN02 – Vínculos Essenciais:** Todo Pedido deve, obrigatoriamente, estar associado a um Cliente, um Vendedor, e possuir um vínculo com uma Nota Fiscal e uma Transportadora para a logística.
- **RN03 – Rastreamento e Logística:** A expedição de um Pedido é registrada no Romaneio, que contém dados de envio como motorista e placa do veículo. O Romaneio é único para cada Pedido.
- **RN04 – Faturamento:** O Pagamento e a Nota Fiscal são registros que se ligam diretamente a um Pedido, sendo indispensáveis para o fechamento do ciclo de vendas e para a geração de relatórios financeiros.

2.3. Trabalhos Relacionados

Existem muitos estudos sobre modelagem e SGBD, mas este projeto se diferencia por aplicar os conceitos teóricos a um ciclo de negócio completo e robusto. O foco é na gestão da cadeia de vendas e logística, representada pela arquitetura de nove tabelas (Produto, Pedido, Cliente, Vendedor, Item, Pagamento, Nota Fiscal, Romaneio, Transportadora). Esta pesquisa utiliza uma estrutura mais complexa e real, aplicando as técnicas avançadas de consulta (Nield) e utilizando o ambiente PostgreSQL, o que mostra a capacidade de adaptar a teoria de normalização (Rob e Coronel) a um cenário empresarial mais exigente.

3 SISTEMA DE VENDAS

Abstração de Dados

Para a estruturação eficaz do banco de dados de vendas, utiliza-se o método de divisão em três níveis de abstração: conceitual, lógico e físico. Este procedimento garante que os requisitos de negócio do sistema de gestão comercial são traduzidos de forma rigorosa para a arquitetura final de nove tabelas, seguindo as diretrizes de modelagem relacional. O software BrModelo é a ferramenta utilizada para auxiliar o design em todas as fases, incluindo a representação visual do modelo físico. O BrModelo possibilita a criação e visualização das nove entidades principais (Item, Pedido, Vendedor, Cliente, Romaneio, Transportadora, Pagamento, Nota Fiscal e Produto) e de seus relacionamentos. Esta representação gráfica serve como base consolidada para a etapa de codificação e implementação do modelo físico no SGBD PostgreSQL (DB Fiddle).

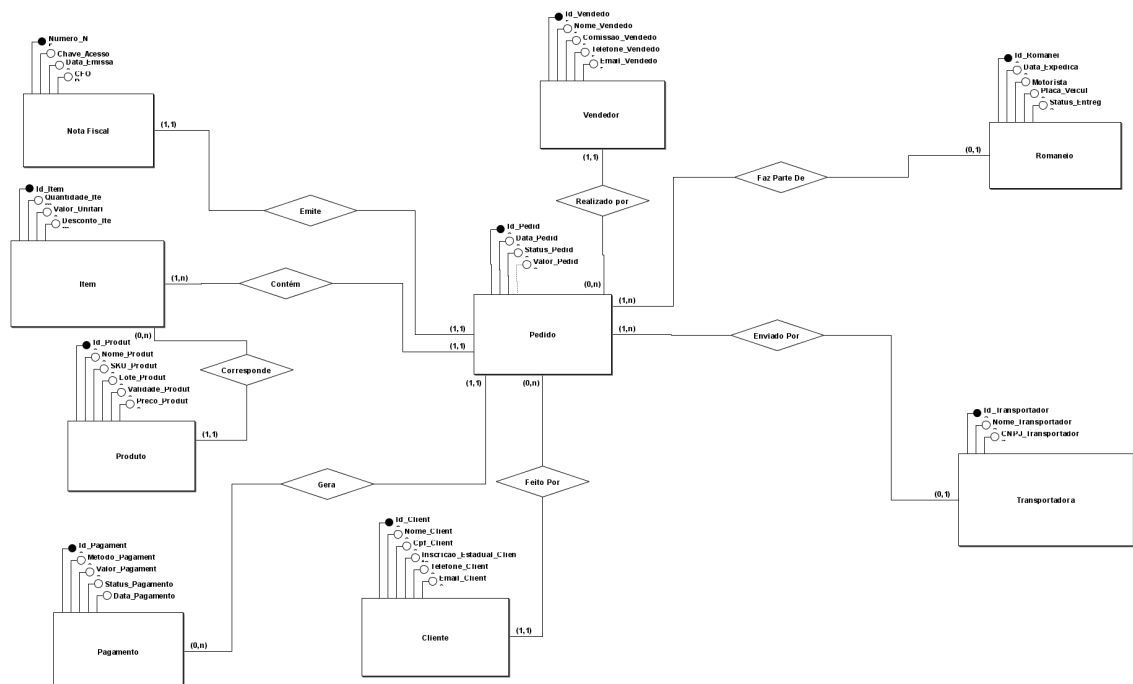


Figura 1 – Modelo Conceitual (BrModelo)

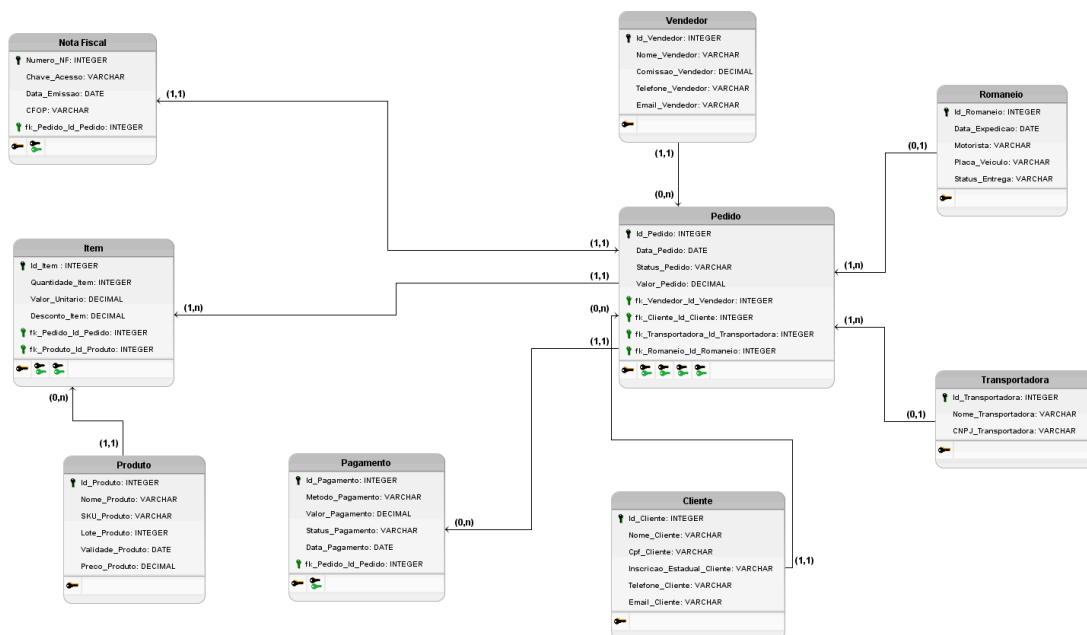


Figura 2 – Modelo Lógico (BrModelo)

```

2.
3. CREAT TABLE Item (
4.   Id_Item INTEGER PRIMARY KEY,
5.   Quantidade_Item INTEGER,
6.   Valor_Unitario DECIMAL,
7.   Desconto_Item DECIMAL,
8.   fk_Pedido_Id_Pedido INTEGER,
9.   fk_Produto_Id_Produto INTEGER
10. );
11.
12. CREAT TABLE Pedido (
13.   Id_Pedido INTEGER PRIMARY KEY,
14.   Data_Pedido DATE,
15.   Status_Pedido VARCHAR,
16.   Valor_Pedido DECIMAL,
17.   fk_Vendedor_Id_Vendedor INTEGER,
18.   fk_Cliente_Id_Cliente INTEGER,
19.   fk_Transportadora_Id_Transportadora INTEGER,
20.   fk_Romaneio_Id_Romaneio INTEGER
21. );
22.
23. CREAT TABLE Vendedor (
24.   Id_Vendedor INTEGER PRIMARY KEY,
25.   Nome_Vendedor VARCHAR,
26.   Comissao_Vendedor DECIMAL,
27.   Telefone_Vendedor VARCHAR,
28.   Email_Vendedor VARCHAR
29. );
30.
31. CREAT TABLE Cliente (
32.   Id_Cliente INTEGER PRIMARY KEY,
33.   Nome_Cliente VARCHAR,
34.   Cpf_Cliente VARCHAR,
35.   Inscricao_Estadual_Cliente VARCHAR,
36.   Telefone_Cliente VARCHAR,
37.   Email_Cliente VARCHAR
38. );
39.
40. CREAT TABLE Romaneio (
41.   Id_Romaneio INTEGER PRIMARY KEY,
42.   Data_Expedicao DATE,
43.   Motorista VARCHAR,
44.   Placa_Veiculo VARCHAR,
45.   Status_Entrega VARCHAR
46. );
47.
48. CREAT TABLE Transportadora (
49.   Id_Transportadora INTEGER PRIMARY KEY,
50.   Nome_Transportadora VARCHAR,
51.   CNPJ_Transportadora VARCHAR
52. );
53.

```

Figura 3 – Modelo Físico (BrModelo)

```

53.
54. CREAT TABLE Pagamento (
55.   Id_Pagamento INTEGER PRIMARY KEY,
56.   Metodo_Pagamento VARCHAR,
57.   Valor_Pagamento DECIMAL,
58.   Status_Pagamento VARCHAR,
59.   Data_Pagamento DATE,
60.   fk_Pedido_Id_Pedido INTEGER
61. );
62.
63. CREAT TABLE Nota Fiscal (
64.   Numero_NF INTEGER PRIMARY KEY,
65.   Chave_Acesso VARCHAR,
66.   Data_Emissao DATE,
67.   CFOP VARCHAR,
68.   fk_Pedido_Id_Pedido INTEGER
69. );
70.
71. CREAT TABLE Produto (
72.   Id_Produto INTEGER PRIMARY KEY,
73.   Nome_Produto VARCHAR,
74.   SKU_Produto VARCHAR,
75.   Lote_Produto INTEGER,
76.   Validade_Produto DATE,
77.   Preco_Produto DECIMAL
78. );
79.
80. ALTE TABLE Item ADD CONSTRAINT FK_Item_2
81.   FOREIGN KEY (fk_Pedido_Id_Pedido)
82.   REFERENCES Pedido (Id_Pedido)
83.   ON DELETE RESTRICT;
84.
85. ALTE TABLE Item ADD CONSTRAINT FK_Item_3
86.   FOREIGN KEY (fk_Produto_Id_Produto)
87.   REFERENCES Produto (Id_Produto)
88.   ON DELETE CASCADE;
89.
90. ALTE TABLE Pedido ADD CONSTRAINT FK_Pedido_2
91.   FOREIGN KEY (fk_Vendedor_Id_Vendedor)
92.   REFERENCES Vendedor (Id_Vendedor)
93.   ON DELETE CASCADE;
94.
95. ALTE TABLE Pedido ADD CONSTRAINT FK_Pedido_3
96.   FOREIGN KEY (fk_Cliente_Id_Cliente)
97.   REFERENCES Cliente (Id_Cliente)
98.   ON DELETE CASCADE;
99.
100. ALTE TABLE Pedido ADD CONSTRAINT FK_Pedido_4
101.   FOREIGN KEY (fk_Transportadora_Id_Transportadora)
102.   REFERENCES Transportadora (Id_Transportadora)
103.   ON DELETE CASCADE;
104.
105. ALTE TABLE Pedido ADD CONSTRAINT FK_Pedido_5
106.   FOREIGN KEY (fk_Romaneio_Id_Romaneio)
107.   REFERENCES Romaneio (Id_Romaneio)
108.   ON DELETE CASCADE;
109.
110. ALTE TABLE Pagamento ADD CONSTRAINT FK_Pagamento_2
111.   FOREIGN KEY (fk_Pedido_Id_Pedido)
112.   REFERENCES Pedido (Id_Pedido)
113.   ON DELETE CASCADE;
114.

```

Figura 4 – Modelo Físico (BrModelo)

```

14.
15. ALTER TABLE Nota Fiscal ADD CONSTRAINT FK_Nota Fiscal_2
16. FOREIGN KEY (fk_Pedido_Id_Pedido)
17. REFERENCES Pedido (Id_Pedido)
18. ON DELETE RESTRICT;

```

Figura 5 – Modelo Físico (BrModelo)

Criação de Tabelas e Inserção de Dados (PostgreSQL)

```

CREATE TABLE Item (
    Id_Item SERIAL PRIMARY KEY,
    Quantidade_Item INTEGER,
    Valor_Unitario DECIMAL(10,2),
    Desconto_Item DECIMAL(10,2),
    fk_Pedido_Id_Pedido INTEGER,
    fk_Produto_Id_Produto INTEGER
);

CREATE TABLE Pedido (
    Id_Pedido SERIAL PRIMARY KEY,
    Data_Pedido DATE,
    Status_Pedido VARCHAR(50),
    Valor_Pedido DECIMAL(10,2),
    fk_Vendedor_Id_Vendedor INTEGER,
    fk_Cliente_Id_Cliente INTEGER,
    fk_Transportadora_Id_Transportadora INTEGER,
    fk_Romaneio_Id_Romaneio INTEGER
);

CREATE TABLE Vendedor (
    Id_Vendedor SERIAL PRIMARY KEY,
    Nome_Vendedor VARCHAR(100),
    Comissao_Vendedor DECIMAL(5,2),
    Telefone_Vendedor VARCHAR(20),
    Email_Vendedor VARCHAR(100)
);

CREATE TABLE Cliente (
    Id_Cliente SERIAL PRIMARY KEY,
    Nome_Cliente VARCHAR(100),
    Cpf_Cliente VARCHAR(14),
    Inscricao_Estadual_Cliente VARCHAR(20),
    Telefone_Cliente VARCHAR(20),
    Email_Cliente VARCHAR(100)
);

CREATE TABLE Romaneio (

```



```

        Id_Romaneio SERIAL PRIMARY KEY,
        Data_Expedicao DATE,
        Motorista VARCHAR(100),
        Placa_Veiculo VARCHAR(10),
        Status_Entrega VARCHAR(50)
    );

CREATE TABLE Transportadora (
    Id_Transportadora SERIAL PRIMARY KEY,
    Nome_Transportadora VARCHAR(100),
    CNPJ_Transportadora VARCHAR(18)
);

CREATE TABLE Pagamento (
    Id_Pagamento SERIAL PRIMARY KEY,
    Metodo_Pagamento VARCHAR(50),
    Valor_Pagamento DECIMAL(10,2),
    Status_Pagamento VARCHAR(50),
    Data_Pagamento DATE,
    fk_Pedido_Id_Pedido INTEGER
);

CREATE TABLE Nota_Fiscal (
    Numero_NF SERIAL PRIMARY KEY,
    Chave_Acesso VARCHAR(44),
    Data_Emissao DATE,
    CFOP VARCHAR(10),
    fk_Pedido_Id_Pedido INTEGER
);

CREATE TABLE Produto (
    Id_Produto SERIAL PRIMARY KEY,
    Nome_Produto VARCHAR(100),
    SKU_Produto VARCHAR(50),
    Lote_Produto INTEGER,
    Validade_Produto DATE,
    Preco_Produto DECIMAL(10,2)
);

ALTER TABLE Item ADD CONSTRAINT FK_Item_2
    FOREIGN KEY (fk_Pedido_Id_Pedido)
    REFERENCES Pedido (Id_Pedido)
    ON DELETE CASCADE;

ALTER TABLE Item ADD CONSTRAINT FK_Item_3
    FOREIGN KEY (fk_Produto_Id_Produto)
    REFERENCES Produto (Id_Produto)
    ON DELETE RESTRICT;

ALTER TABLE Pedido ADD CONSTRAINT FK_Pedido_2
    FOREIGN KEY (fk_Vendedor_Id_Vendedor)

```

```

REFERENCES Vendedor (Id_Vendedor)
ON DELETE RESTRICT;

ALTER TABLE Pedido ADD CONSTRAINT FK_Pedido_3
FOREIGN KEY (fk_Cliente_Id_Cliente)
REFERENCES Cliente (Id_Cliente)
ON DELETE RESTRICT;

ALTER TABLE Pedido ADD CONSTRAINT FK_Pedido_4
FOREIGN KEY (fk_Transportadora_Id_Transportadora)
REFERENCES Transportadora (Id_Transportadora)
ON DELETE RESTRICT;

ALTER TABLE Pedido ADD CONSTRAINT FK_Pedido_5
FOREIGN KEY (fk_Romaneio_Id_Romaneio)
REFERENCES Romaneio (Id_Romaneio)
ON DELETE SET NULL;

ALTER TABLE Pagamento ADD CONSTRAINT FK_Pagamento_2
FOREIGN KEY (fk_Pedido_Id_Pedido)
REFERENCES Pedido (Id_Pedido)
ON DELETE CASCADE;

ALTER TABLE Nota_Fiscal ADD CONSTRAINT FK_Nota_Fiscal_2
FOREIGN KEY (fk_Pedido_Id_Pedido)
REFERENCES Pedido (Id_Pedido)
ON DELETE CASCADE;

```

Algoritmo 1 – Criação das Tabelas (PostgreSQL)

```

INSERT INTO Cliente (Nome_Cliente, Cpf_Cliente, Inscricao_Estadual_Cliente,
Telefone_Cliente, Email_Cliente) VALUES
('Mundo Verde Taubaté', '111.111.111-11', '123456789', '(12) 3632-1111',
'compras@mundoverdetaubate.com.br'),
('Supermercado Semar', '222.222.222-22', '987654321', '(12) 3622-2222',
'compras@semar.com.br'),
('Burger King Taubaté', '333.333.333-33', '123456789', '(12) 3632-3333',
'compras@bktaubate.com'),
('McDonalds', '444.444.444-44', '987654321', '(12) 3632-4444',
'compras@mcdonalds.com'),
('Empório Fit Vale', '555.555.555-55', '112233445', '(12) 3621-5555',
'contato@emporiofitvale.com.br'),
('Distribuidora Saudável', '666.666.666-66', '555666777', '(11)
4002-6666', 'logistica@distribuidorasaudavel.com.br'),
('Nutri Store Suplementos', '777.777.777-77', '999888777', '(11)
4002-7777', 'compras@nutristore.com.br'),
('Atacado S.A.', '888.888.888-88', '111222333', '(11) 3000-8888',
'fornecedores@atacado.com.br'),

```

```

        ('Armazém Natural', '999.999.999-99', '444555666', '(21) 2000-9999',
'compras@armazemnatural.com.br'),
        ('Supermercado Shibata', '000.000.000-00', '777888999', '(12) 3624-0000',
'compras@shibata.com.br');

INSERT INTO Vendedor (Nome_Vendedor, Comissao_Vendedor, Telefone_Vendedor,
Email_Vendedor) VALUES
        ('Thiago Alvarenga', 5.50, '(12) 98111-0001',
'thiago.alvarenga@mrstaste.com.br'),
        ('Marcelo Nogueira', 4.00, '(12) 98111-0002',
'marcelo.nogueira@mrstaste.com.br'),
        ('Camila Rezende', 6.00, '(12) 98111-0003',
'camila.rezende@mrstaste.com.br'),
        ('Rodrigo Peixoto', 5.00, '(12) 98111-0004',
'rodrigo.peixoto@mrstaste.com.br'),
        ('Letícia Viana', 4.50, '(11) 97222-0005',
'leticia.viana@mrstaste.com.br'),
        ('Fernando Maciel', 5.50, '(11) 97222-0006',
'fernando.maciel@mrstaste.com.br'),
        ('Amanda Goulart', 6.50, '(21) 96333-0007',
'amanda.goulart@mrstaste.com.br'),
        ('Diego Farias', 5.00, '(21) 96333-0008',
'diego.farias@mrstaste.com.br'),
        ('Bruno Meireles', 4.00, '(31) 95444-0009',
'bruno.meireles@mrstaste.com.br'),
        ('Tatiana Silveira', 5.50, '(31) 95444-0010',
'tatiana.silveira@mrstaste.com.br');

INSERT INTO Transportadora (Nome_Transportadora, CNPJ_Transportadora) VALUES
        ('Pássaro Marron Logística', '11.111.111/0001-11'),
        ('Correios Sedex', '22.222.222/0001-22'),
        ('TransTaubaté Expresso', '33.333.333/0001-33'),
        ('Loggi', '44.444.444/0001-44'),
        ('Jadlog', '55.555.555/0001-55'),
        ('Braspress', '66.666.666/0001-66'),
        ('TNT Mercurio', '77.777.777/0001-77'),
        ('Azul Cargo', '88.888.888/0001-88'),
        ('Latam Cargo', '99.999.999/0001-99'),
        ('Total Express', '00.000.000/0001-00');

INSERT INTO Romaneio (Data_Expedicao, Motorista, Placa_Veiculo,
Status_Entrega) VALUES
        ('2026-05-30', 'Carlos Silva', 'ABC-1234', 'Em Rota'),
        ('2026-05-31', 'José Souza', 'XYZ-9876', 'Pendente'),
        ('2026-06-01', 'Marcos Oliveira', 'QWE-5555', 'Pendente'),
        ('2026-05-28', 'Antonio Costa', 'ASD-1111', 'Entregue'),
        ('2026-05-29', 'Francisco Santos', 'ZXC-2222', 'Entregue'),
        ('2026-06-02', 'Paulo Mendes', 'FGH-3333', 'Pendente'),
        ('2026-06-03', 'Luis Roberto', 'VBN-4444', 'Pendente'),
        ('2026-05-27', 'Fernando Lima', 'RTY-6666', 'Entregue'),
        ('2026-06-04', 'Ricardo Alves', 'UIO-7777', 'Pendente'),

```

```

('2026-06-05', 'Marcelo Pereira', 'JKL-8888', 'Pendente');

INSERT INTO Produto (Nome_Produto, SKU_Produto, Lote_Produto,
Validade_Produto, Preco_Produto) VALUES
('Maionese Tradicional Zero', 'MRS-MAIO-01', 1001, '2027-01-01', 18.90),
('Ketchup Zero Sódio', 'MRS-KET-02', 1002, '2027-02-01', 15.50),
('Molho Barbecue Zero', 'MRS-BBQ-03', 1003, '2027-03-01', 22.90),
('Maionese com Alho', 'MRS-MAAL-04', 1004, '2027-04-01', 19.90),
('Calda de Chocolate Zero', 'MRS-CHOC-05', 2001, '2028-01-01', 24.50),
('Calda de Caramelo Zero', 'MRS-CARA-06', 2002, '2028-02-01', 24.50),
('Molho Shoyu Zero Sódio', 'MRS-SHOY-07', 3001, '2027-06-01', 16.90),
('Molho Mostarda Zero', 'MRS-MOST-08', 3002, '2027-07-01', 14.90),
('Creme de Avelã Zero', 'MRS-AVEL-09', 4001, '2028-05-01', 29.90),
('Tempero Frango Zero', 'MRS-TEMP-10', 5001, '2029-01-01', 12.50);

INSERT INTO Pedido (Data_Pedido, Status_Pedido, Valor_Pedido,
fk_Vendedor_Id_Vendedor, fk_Cliente_Id_Cliente,
fk_Transportadora_Id_Transportadora, fk_Romaneio_Id_Romaneio) VALUES
('2026-05-25', 'Faturado', 945.00, 1, 3, 1, 1),
('2026-05-26', 'Faturado', 620.00, 2, 4, 2, 2),
('2026-05-27', 'Separacao', 453.00, 3, 5, 3, 3),
('2026-05-28', 'Entregue', 199.00, 4, 1, 4, 4),
('2026-05-29', 'Entregue', 235.00, 5, 2, 5, 5),
('2026-05-30', 'Pendente', 122.50, 6, 6, 6, 6),
('2026-05-30', 'Pendente', 250.00, 7, 7, 7, 7),
('2026-05-24', 'Entregue', 298.00, 8, 8, 8, 8),
('2026-05-31', 'Pendente', 299.00, 9, 9, 9, 9),
('2026-05-31', 'Pendente', 250.00, 10, 10, 10, 10);

INSERT INTO Item (Quantidade_Item, Valor_Unitario, Desconto_Item,
fk_Pedido_Id_Pedido, fk_Produto_Id_Produto) VALUES
(50, 18.90, 0.00, 1, 1), -- 50 x Maionese Tradicional = 945.00
(40, 15.50, 0.00, 2, 2), -- 40 x Ketchup Zero = 620.00
(20, 22.90, 5.00, 3, 3), -- 20 x Barbecue (458) - 5 de desc = 453.00
(10, 19.90, 0.00, 4, 4), -- 10 x Maionese com Alho = 199.00
(10, 24.50, 10.00, 5, 5), -- 10 x Calda Chocolate (245) - 10 de desc =
235.00
(5, 24.50, 0.00, 6, 6), -- 5 x Calda Caramelo = 122.50
(15, 16.90, 3.50, 7, 7), -- 15 x Shoyu (253.50) - 3.50 de desc = 250.00
(20, 14.90, 0.00, 8, 8), -- 20 x Mostarda Zero = 298.00
(10, 29.90, 0.00, 9, 9), -- 10 x Creme de Avelã = 299.00
(20, 12.50, 0.00, 10, 10); -- 20 x Tempero Frango = 250.00

INSERT INTO Pagamento (Metodo_Pagamento, Valor_Pagamento, Status_Pagamento,
Data_Pagamento, fk_Pedido_Id_Pedido) VALUES
('Pix', 945.00, 'Aprovado', '2026-05-25', 1),
('Boleto', 620.00, 'Aguardando', '2026-05-26', 2),
('Cartão de Crédito', 453.00, 'Aprovado', '2026-05-27', 3),
('Pix', 199.00, 'Aprovado', '2026-05-28', 4),
('Cartão de Débito', 235.00, 'Aprovado', '2026-05-29', 5),
('Pix', 122.50, 'Recusado', '2026-05-30', 6),

```

```

('Pix', 250.00, 'Aprovado', '2026-05-30', 7),
('Cartão de Crédito', 298.00, 'Aprovado', '2026-05-24', 8),
('Cartão de Débito', 299.00, 'Aprovado', '2026-05-31', 9),
('Pix', 250.00, 'Aguardando', '2026-05-31', 10);

INSERT INTO Nota_Fiscal (Chave_Acesso, Data_Emissao, CFOP,
fk_Pedido_Id_Pedido) VALUES
('35260511111111000111550010000000011234567890', '2026-05-25', '5102',
1),
('35260522222222000122550010000000021234567890', '2026-05-26', '6102',
2),
('35260533333333000133550010000000031234567890', '2026-05-27', '5102',
3),
('35260544444444000144550010000000041234567890', '2026-05-28', '5102',
4),
('35260555555555000155550010000000051234567890', '2026-05-29', '6102',
5),
('35260566666666000166550010000000061234567890', '2026-05-30', '5102',
6),
('35260577777777000177550010000000071234567890', '2026-05-30', '5102',
7),
('35260588888888000188550010000000081234567890', '2026-05-24', '6102',
8),
('35260599999999000199550010000000091234567890', '2026-05-31', '5102',
9),
('352605000000000000100550010000000101234567890', '2026-05-31', '5102',
10);

```

Algoritmo 2 – Inserção de Dados (PostgreSQL)

3.3 Consultas Propostas

Resolução prática das atividades de validação do sistema de vendas alimentícias.

O desenvolvimento prático contempla a aplicação da linguagem SQL para solucionar trinta problemas de negócio e relatórios gerenciais baseados no cenário logístico e financeiro de uma empresa de vendas alimentícias, integrando os fundamentos

conceituais de Peter Rob e Carlos Coronel, associados às técnicas de consulta de Thomas Nield. As atividades realizadas abrangem a totalidade das trinta consultas avançadas propostas para o escopo da disciplina de Banco de Dados II. Cada resolução apresentada neste relatório segue uma sequência lógica de complexidade crescente, utilizando o ambiente PostgreSQL para a validação dos resultados obtidos.

Resolução prática das atividades de validação do sistema de clínica veterinária. O desenvolvimento prático contempla a aplicação da linguagem SQL para solucionar trinta problemas de negócio e relatórios gerenciais baseados no cenário de uma clínica veterinária, integrando os fundamentos conceituais de Peter Rob e Carlos Coronel, associados às técnicas de consulta de Thomas Nield. As atividades realizadas abrangem a totalidade das trinta consultas avançadas propostas para o escopo da disciplina de Banco de Dados II. Cada resolução apresentada neste relatório segue uma sequência lógica de complexidade crescente, utilizando o ambiente DB Fiddle para a validação dos resultados obtidos.

- Consulta 1 -

```
SELECT
    c.Nome_Cliente,
    p.Id_Pedido,
    nf.Chave_Acesso,
    t.Nome_Transportadora,
    r.Status_Entrega,
    r.Data_Expedicao
FROM Pedido p
INNER JOIN Cliente c ON p.fk_Cliente_Id_Cliente = c.Id_Cliente
INNER JOIN Nota_Fiscal nf ON nf.fk_Pedido_Id_Pedido = p.Id_Pedido
INNER JOIN Transportadora t ON p.fk_Transportadora_Id_Transportadora =
t.Id_Transportadora
INNER JOIN Romaneio r ON p.fk_Romaneio_Id_Romaneio = r.Id_Romaneio
WHERE r.Status_Entrega != 'Entregue'
ORDER BY r.Data_Expedicao ASC;
```

Algoritmo 3 – Consulta 1 (PostgreSQL)

nome_cliente	id_pedido	chave_acesso	nome_transportadora	status_entrega	data_expedicao
Burger King Taubaté	1	352605111111000111550010000000011234567890	Pássaro Marron Logística	Em Rota	2026-05-30
McDonalds	2	3526052222220001122550010000000001234567890	Correios Sedex	Pendente	2026-05-31
Empório Fil Vale	3	352605333333000113550010000000001234567890	TransTaubaté Expresso	Pendente	2026-06-01
Distribuidora Saudável	6	35260566666600011665500100000000061234567890	Braspress	Pendente	2026-06-02
Nutri Store Suplementos	7	3526057777770001175500100000000071234567890	TNT Mercurio	Pendente	2026-06-03
Armazém Natural	9	35260599999900011995500100000000091234567890	Latam Cargo	Pendente	2026-06-04
Supermercado Shibata	10	3526050000000000011055500100000000101234567890	Total Express	Pendente	2026-06-05

Figura 6 – Consulta 1 (PostgreSQL)

- Consulta 2 -

```
SELECT
    prod.Nome_Produto,
    SUM(i.Quantidade_Item * i.Valor_Unitario) AS Receita_Gerada,
    RANK() OVER (ORDER BY SUM(i.Quantidade_Item * i.Valor_Unitario)
DESC) AS Posicao_Ranking
FROM Item i
INNER JOIN Produto prod ON i.fk_Produto_Id_Produto = prod.Id_Produto
GROUP BY prod.Nome_Produto;
}
```

Algoritmo 4 – Consulta 2 (PostgreSQL)

nome_produto	receita_gerada	posicao_ranking
Maconese Tradicional Zero	945.00	1
Ketchup Zero Sódio	620.00	2
Molho Barbecue Zero	458.00	3
Creme de Avelã Zero	299.00	4
Molho Mostarda Zero	298.00	5
Molho Shoyu Zero Sódio	253.50	6
Tempero Frango Zero	250.00	7
Calda de Chocolate Zero	245.00	8
Maconese com Alho	199.00	9
Calda de Caramelo Zero	122.50	10

Figura 7 – Consulta 2 (Microsoft SQL Server)

- Consulta 3 -

```
SELECT
    v.Nome_Vendedor,
    COUNT(p.Id_Pedido) AS Total_Pedidos_Feitos,
    SUM(p.Valor_Pedido) AS Total_Vendido,
    (SUM(p.Valor_Pedido) * (v.Comissao_Vendedor / 100)) AS
Comissao_A_Receber
FROM Vendedor v
INNER JOIN Pedido p ON p.fk_Vendedor_Id_Vendedor = v.Id_Vendedor
WHERE p.Status_Pedido = 'Faturado' OR p.Status_Pedido = 'Entregue'
```

```
GROUP BY v.Nome_Vendedor, v.Comissao_Vendedor
ORDER BY Total_Vendido DESC;
```

Algoritmo 5 – Consulta 3 (PostgreSQL)

nome_vendedor	total_pedidos_feitos	total_vendido	comissao_a_receber
Thiago Alvarenga	1	945.00	51.975000000000000000000000
Marcelo Nogueira	1	620.00	24.800000000000000000000000
Diego Farias	1	298.00	14.900000000000000000000000
Letícia Viana	1	235.00	10.575000000000000000000000
Rodrigo Peixoto	1	199.00	9.950000000000000000000000

Figura 8 – Consulta 3 (PostgreSQL)

- Consulta 4 -

```
#SELECT
    c.Nome_Cliente,
    AVG(p.Valor_Pedido) AS Ticket_Medio_Cliente
FROM Cliente c
INNER JOIN Pedido p ON p.fk_Cliente_Id_Cliente = c.Id_Cliente
GROUP BY c.Nome_Cliente
HAVING AVG(p.Valor_Pedido) > (
    SELECT AVG(Valor_Pedido) FROM Pedido
);
```

Algoritmo 6 – Consulta 4 (PostgreSQL)

nome_cliente	ticket_medio_cliente
Empório Fa Vale	453.0000000000000000000000
Burger King Taubaté	945.0000000000000000000000
McDonalds	620.0000000000000000000000

Figura 9 – Consulta 4 (PostgreSQL)

- Consulta 5 -

```
WITH Analise_Descontos AS (
    SELECT
        fk_Produto_Id_Produto,
        SUM(Desconto_Item) AS Total_Descontado,
        SUM(Quantidade_Item) AS Volume_Vendido
    FROM Item
    WHERE Desconto_Item > 0
    GROUP BY fk_Produto_Id_Produto
```



```

)
SELECT
    prod.Nome_Produto,
    ad.Total_Descontado,
    ad.Volume_Vendido,
    (ad.Total_Descontado / ad.Volume_Vendido) AS
Desconto_Medio_Por_Unidade
FROM Analise_Descontos ad
INNER JOIN Produto prod ON ad.fk_Produto_Id_Produto = prod.Id_Produto
ORDER BY ad.Total_Descontado DESC;

```

Algoritmo 7 – Consulta 5 (PostgreSQL)

nome_produto	total_descontado	volume_vendido	desconto_medio_por_unidade
Caldá de Chocolate Zero	10.00	10	1.00000000000000000000
Molho Barbecue Zero	5.00	20	0.25000000000000000000
Molho Shoyu Zero Sódio	3.50	15	0.23333333333333333333

Figura 10 – Consulta 5 (PostgreSQL)

- Consulta 6 -

```

SELECT
    Id_Produto,
    Nome_Produto,
    Preço_Produto
FROM Produto
WHERE Nome_Produto ILIKE '%Zero%'
ORDER BY Preço_Produto DESC;

```

Algoritmo 8 – Consulta 6 (PostgreSQL)

id_produto	nome_produto	preco_produto
9	Creme de Avelã Zero	29.90
6	Caldê de Caramelo Zero	24.50
5	Caldê de Chocolate Zero	24.50
3	Molho Barbecue Zero	22.90
1	Maionese Tradicional Zero	18.90
7	Molho Shoyu Zero Sódio	16.90
2	Ketchup Zero Sódio	15.50
8	Molho Mostarda Zero	14.90
10	Tempero Frango Zero	12.50

Figura 11 – Consulta 6 (PostgreSQL)

- Consulta 7 -

```
SELECT
    fk_Pedido_Id_Pedido AS Pedido,
    fk_Produto_Id_Produto AS Produto,
    Valor_Unitario,
    Desconto_Item,
    ROUND((Desconto_Item / (Quantidade_Item * Valor_Unitario)) * 100,
2) AS Porcentagem_Desconto
FROM Item
WHERE Desconto_Item > 0;
```

Algoritmo 9 – Consulta 7 (PostgreSQL)

pedido	produto	valor_unitario	desconto_item	porcentagem_desconto
3	3	22.90	5.00	1.09
5	5	24.50	10.00	4.08
7	7	16.90	3.50	1.38

Figura 12 – Consulta 7 (PostgreSQL)

- Consulta 8 -

```
SELECT
    Id_Pedido,
    Data_Pedido,
    Valor_Pedido,
    CASE
        WHEN Valor_Pedido > 500 THEN 'Pedido Grande (Premium)'
        WHEN Valor_Pedido BETWEEN 200 AND 500 THEN 'Pedido Médio'
        ELSE 'Pedido Pequeno'
    END AS Categoria_Venda
```

```
FROM Pedido
ORDER BY Valor_Pedido DESC;
```

Algoritmo 10 – Consulta 8 (PostgreSQL)

Id_pedido	data_pedido	valor_pedido	categoria_venda
1	2026-05-25	945.00	Pedido Grande (Premium)
2	2026-05-26	620.00	Pedido Grande (Premium)
3	2026-05-27	453.00	Pedido Médio
9	2026-05-31	299.00	Pedido Médio
8	2026-05-24	298.00	Pedido Médio
10	2026-05-31	250.00	Pedido Médio
7	2026-05-30	250.00	Pedido Médio
5	2026-05-29	235.00	Pedido Médio
4	2026-05-28	199.00	Pedido Pequeno
6	2026-05-30	122.50	Pedido Pequeno

Figura 13 –Consulta 8 (PostgreSQL)

● Consulta 9 -

```
SELECT
    Id_Pedido,
    Valor_Pedido,
    ROUND((Valor_Pedido / (SELECT SUM(Valor_Pedido) FROM Pedido)) *
100, 2) AS Porcentagem_Do_Faturamento_Total
FROM Pedido
ORDER BY Porcentagem_Do_Faturamento_Total DESC;
```

Algoritmo 11 – Consulta 9 (PostgreSQL)

Id_pedido	valor_pedido	porcentagem_do_faturamento_total
1	945.00	25.74
2	620.00	16.89
3	453.00	12.34
9	299.00	8.14
8	298.00	8.12
10	250.00	6.81
7	250.00	6.81
5	235.00	6.40
4	199.00	5.42
6	122.50	3.34

Figura 14 – Consulta 9 (PostgreSQL)

- Consulta 10 -

```
SELECT
    SUBSTRING(Email_Cliente FROM POSITION('@' IN Email_Cliente) + 1) AS
    Dominio_Email,
    COUNT(*) AS Total_Clientes
FROM Cliente
GROUP BY SUBSTRING(Email_Cliente FROM POSITION('@' IN Email_Cliente) +
1)
ORDER BY Total_Clientes DESC;
```

Algoritmo 12 – Consulta 10 (PostgreSQL)

dominio_email	total_clientes
shibata.com.br	1
bkaubate.com	1
distribuidorasaudavel.com.br	1
nutristore.com.br	1
atacado.com.br	1
emporiofivale.com.br	1
mcdonalds.com	1
semar.com.br	1
armazemnatural.com.br	1
mundoverdeaubate.com.br	1

Figura 15 – Consulta 10 (PostgreSQL)

- Consulta 11 -

```
SELECT
    Id_Pedido,
    Data_Pedido,
    Valor_Pedido AS Faturamento_Atual,
    LAG(Valor_Pedido) OVER (ORDER BY Data_Pedido, Id_Pedido) AS
    Faturamento_Anterior,
    Valor_Pedido - LAG(Valor_Pedido) OVER (ORDER BY Data_Pedido,
    Id_Pedido) AS Diferenca_Em_Reais
FROM Pedido
WHERE Status_Pedido IN ('Faturado', 'Entregue');
```

Algoritmo 13 – Consulta 11 (PostgreSQL)

id_pedido	data_pedido	faturamento_atual	faturamento_anterior	diferenca_em_reais
8	2026-05-24	298.00	null	null
1	2026-05-25	945.00	298.00	647.00
2	2026-05-26	620.00	945.00	-325.00
4	2026-05-28	199.00	620.00	-421.00
5	2026-05-29	235.00	199.00	36.00

Figura 16 – Consulta 11 (PostgreSQL)

- Consulta 12 -

```
SELECT
    c.Nome_Cliente,
    SUM(i.Quantidade_Item) AS Frascos_Comprados
FROM Cliente c
INNER JOIN Pedido p ON c.Id_Cliente = p.fk_Cliente_Id_Cliente
INNER JOIN Item i ON p.Id_Pedido = i.fk_Pedido_Id_Pedido
GROUP BY c.Nome_Cliente
HAVING SUM(i.Quantidade_Item) > 15
ORDER BY Frascos_Comprados DESC;
```

Algoritmo 14 – Consulta 12 (PostgreSQL)

nome_cliente	frascos_comprados
Burger King Taubaté	50
McDonalds	40
Empório Fit Vale	20
Supermercado Shibata	20
Alacadio S.A.	20

Figura 17 – Consulta 12 (PostgreSQL)

- Consulta 13 -

```
SELECT
    COALESCE(Metodo_Pagamento, 'TOTAL GERAL ARRECADADO') AS
Tipo_Pagamento,
    SUM(Valor_Pagamento) AS Receita_Total
FROM Pagamento
GROUP BY ROLLUP(Metodo_Pagamento)
ORDER BY Receita_Total DESC;
```

Algoritmo 15 – Consulta 13 (PostgreSQL)

tipo_pagamento	receita_total
TOTAL GERAL ARRECADADO	3671.50
Pix	1766.50
Cartão de Crédito	751.00
Boleto	620.00
Cartão de Débito	534.00

Figura 18 –Consulta 13 (PostgreSQL)

- Consulta 14 -

```
SELECT
    Metodo_Pagamento,
    Valor_Pagamento,
    fk_Pedido_Id_Pedido,
    ROW_NUMBER() OVER (PARTITION BY Metodo_Pagamento ORDER BY
Valor_Pagamento DESC) AS Ranking_Dentro_Do_Metodo
FROM Pagamento;
```

Algoritmo 16 – Consulta 14 (PostgreSQL)

metodo_pagamento	valor_pagamento	fk_pedido_id_pedido	ranking_dentro_do_metodo
Boleto	620.00	2	1
Cartão de Crédito	453.00	3	1
Cartão de Crédito	298.00	8	2
Cartão de Débito	299.00	9	1
Cartão de Débito	235.00	5	2
Pix	945.00	1	1
Pix	250.00	10	2
Pix	250.00	7	3
Pix	199.00	4	4
Pix	122.50	6	5

Figura 19 – Consulta 14 (PostgreSQL)

- Consulta 15 -

```
SELECT 'Cliente' AS Categoria, Nome_Cliente AS Nome, Email_Cliente AS
Contato FROM Cliente
UNION ALL
1 SELECT 'Vendedor' AS Categoria, Nome_Vendedor AS Nome, Email_Vendedor
AS Contato FROM Vendedor
ORDER BY Categoria, Nome;
```

Algoritmo 17 – Consulta 15 (PostgreSQL)

categoria	nome	contato
Cliente	Amazon Natural	compras@amazonnatural.com.br
Cliente	Atacadão S.A.	fornecedores@atacaoaes.com.br
Cliente	Burger King Taubaté	compras@bksaubate.com
Cliente	Distribuidora Saudável	logistica@distribuidorasaudavel.com.br
Cliente	Empório Fit Vale	contato@emporiofitvale.com.br
Cliente	McDonalds	compras@mcDonalds.com
Cliente	Mundo Verde Taubaté	compras@mundovertetaubate.com.br
Cliente	Nutri Store Suplementos	compras@nutristore.com.br
Cliente	Supermercado Semar	compras@semar.com.br
Cliente	Supermercado Shibata	compras@shibata.com.br
Vendedor	Amanda Goulart	amanda.goulart@mrstaste.com.br
Vendedor	Bruno Meireles	bruno.meireles@mrstaste.com.br
Vendedor	Camila Rezende	camila.rezende@mrstaste.com.br
Vendedor	Diego Farias	diego.farias@mrstaste.com.br
Vendedor	Fernando Maciel	fernando.maciell@mrstaste.com.br
Vendedor	Letícia Viana	leticia.viana@mrstaste.com.br
Vendedor	Marcelo Nogueira	marcelo.nogueira@mrstaste.com.br
Vendedor	Rodrigo Perotto	rodrigo.perotto@mrstaste.com.br
Vendedor	Tatiana Silveira	tatiana.silveira@mrstaste.com.br
Vendedor	Thiago Alvarenga	thiago.alvarenga@mrstaste.com.br

Figura 20 – Consulta 15 (PostgreSQL)

- Consulta 16 -

```
SELECT
    t.Nome_Transportadora,
    COUNT(r.Id_Romaneio) AS Total_Viagens,
    COUNT(r.Id_Romaneio) FILTER (WHERE r.Status_Entrega = 'Entregue')
AS Viagens_Finalizadas
FROM Transportadora t
INNER JOIN Pedido p ON t.Id_Transportadora =
p.fk_Transportadora_Id_Transportadora
INNER JOIN Romaneio r ON p.fk_Romaneio_Id_Romaneio = r.Id_Romaneio
GROUP BY t.Nome_Transportadora;
```

Algoritmo 18 –Consulta 16 (PostgreSQL)

nome_transportadora	total_viagens	viagens_finalizadas
TNT Mercurio	1	0
Loggi	1	1
Jadlog	1	1
TransTaubaté Expresso	1	0
Pássaro Marron Logística	1	0
Braspress	1	0
Correios Sedex	1	0
Latam Cargo	1	0
Total Express	1	0
Azul Cargo	1	1

Figura 21 – Consulta 16 (PostgreSQL)

- Consulta 17 -

```
SELECT
    EXTRACT(DAY FROM Data_Emissao) AS Dia_Do_Mes,
    COUNT(Numero_NF) AS Quantidade_De_Notas,
    SUM(p.Valor_Pedido) AS Faturamento_Do_Dia
FROM Nota_Fiscal nf
INNER JOIN Pedido p ON nf.fk_Pedido_Id_Pedido = p.Id_Pedido
GROUP BY EXTRACT(DAY FROM Data_Emissao)
ORDER BY Faturamento_Do_Dia DESC;
```

Algoritmo 19 – Consulta 17 (PostgreSQL)

dia_do_mes	quantidade_de_notas	faturamento_do_dia
25	1	945.00
26	1	620.00
31	2	549.00
27	1	453.00
30	2	372.50
24	1	298.00
29	1	235.00
28	1	199.00

Figura 22 –Consulta 17 (PostgreSQL)

- Consulta 18 -

```
SELECT
    p.Id_Pedido,
    prod.Nome_Produto,
    prod.Preco_Produto AS Preco_Catalogo,
    i.Valor_Unitario AS Preco_Praticado_Na_Venda,
    (prod.Preco_Produto - i.Valor_Unitario) AS Diferenca_De_Preco
FROM Item i
INNER JOIN Produto prod ON i.fk_Produto_Id_Produto = prod.Id_Produto
INNER JOIN Pedido p ON i.fk_Pedido_Id_Pedido = p.Id_Pedido;
```

Algoritmo 20 – Consulta 18 (PostgreSQL)

id_pedido	nome_produto	preco_catalogo	preco_praticado_na_venda	diferenca_de_preco
1	Maconese Tradicional Zero	18.90	18.90	0.00
2	Ketchup Zero Sódio	15.50	15.50	0.00
3	Molho Barbecue Zero	22.90	22.90	0.00
4	Maconese com Alho	19.90	19.90	0.00
5	Calda de Chocolate Zero	24.50	24.50	0.00
6	Calda de Caramelo Zero	24.50	24.50	0.00
7	Molho Shoyu Zero Sódio	16.90	16.90	0.00
8	Molho Mostarda Zero	14.90	14.90	0.00
9	Creme de Avelã Zero	29.90	29.90	0.00
10	Tempero Frango Zero	12.50	12.50	0.00

Figura 23 – Consulta 18 (PostgreSQL)

- Consulta 19 -

```
SELECT
    Numero_NF,
    Chave_Acesso,
    CFOP,
    REPLACE(REPLACE(CFOP, '5102', 'Venda no Estado'), '6102', 'Venda
    Fora do Estado') AS Descricao_Operacao
FROM Nota_Fiscal;
```

Algoritmo 21 – Consulta 19 (PostgreSQL)

numero_nf	chave_acesso	cfop	descricao_operacao
1	3526051111111000111550010000000011234567890	5102	Venda no Estado
2	3526052222222000122550010000000021234567890	6102	Venda Fora do Estado
3	3526053333333000133550010000000031234567890	5102	Venda no Estado
4	3526054444444000144550010000000041234567890	5102	Venda no Estado
5	3526055555555000155550010000000051234567890	6102	Venda Fora do Estado
6	3526056666666000166550010000000061234567890	5102	Venda no Estado
7	3526057777777000177550010000000071234567890	5102	Venda no Estado
8	3526058888888000188550010000000081234567890	6102	Venda Fora do Estado
9	3526059999999000199550010000000091234567890	5102	Venda no Estado
10	35260500000000000100550010000000101234567890	5102	Venda no Estado

Figura 24 – Consulta 19 (PostgreSQL)

- Consulta 20 -

```
SELECT
    v.Nome_Vendedor,
    COUNT(DISTINCT p.fk_Cliente_Id_Cliente) AS Clientes_Atendidos,
    SUM(i.Quantidade_Item) AS Total_Frascos_Vendidos,
    SUM(p.Valor_Pedido) AS Faturamento_Total
FROM Vendedor v
INNER JOIN Pedido p ON v.Id_Vendedor = p.fk_Vendedor_Id_Vendedor
INNER JOIN Item i ON p.Id_Pedido = i.fk_Pedido_Id_Pedido
GROUP BY v.Nome_Vendedor
```

```
ORDER BY Faturamento_Total DESC;
```

Algoritmo 22 – Consulta 20 (PostgreSQL)

nome_vendedor	clientes_atendidos	total_frascos_vendidos	faturamento_total
Thiago Alvarenga	1	50	945.00
Marcelo Nogueira	1	40	620.00
Camila Rezende	1	20	453.00
Bruno Meireles	1	10	298.00
Diego Farias	1	20	298.00
Amanda Goulart	1	15	250.00
Tatiana Silveira	1	20	250.00
Leticia Viana	1	10	235.00
Rodrigo Peixoto	1	10	199.00
Fernando Maciel	1	5	122.50

Figura 25 – Consulta 20 (PostgreSQL)

- Consulta 21 -

```
WITH Calendario AS (
    SELECT generate_series('2026-05-24'::date, '2026-05-31'::date, '1
day'::interval)::date AS Dia
)
SELECT
    TO_CHAR(c.Dia, 'DD/MM/YYYY') AS Data_Referencia,
    COALESCE(COUNT(p.Id_Pedido), 0) AS Qtd_Pedidos,
    COALESCE(SUM(p.Valor_Pedido), 0.00) AS Faturamento_Total
FROM Calendario c
LEFT JOIN Pedido p ON c.Dia = p.Data_Pedido
GROUP BY c.Dia
ORDER BY c.Dia;
```

Algoritmo 23 – Consulta 21 (PostgreSQL)

data_referencia	qtd_pedidos	faturamento_total
24/05/2026	1	298.00
25/05/2026	1	945.00
26/05/2026	1	620.00
27/05/2026	1	453.00
28/05/2026	1	199.00
29/05/2026	1	235.00
30/05/2026	2	372.50
31/05/2026	2	549.00

Figura 26 – Consulta 21 (PostgreSQL)

- Consulta 22 -

```

SELECT
    t.Nome_Transportadora,
    SUM(p.Valor_Pedido) FILTER (WHERE p.Status_Pedido = 'Faturado') AS
Valor_Faturado,
    SUM(p.Valor_Pedido) FILTER (WHERE p.Status_Pedido = 'Entregue') AS
Valor_Entregue,
    SUM(p.Valor_Pedido) FILTER (WHERE p.Status_Pedido = 'Pendente') AS
Valor_Pendente,
    SUM(p.Valor_Pedido) AS Valor_Total_Carga
FROM Transportadora t
INNER JOIN Pedido p ON t.Id_Transportadora =
p.fk_Transportadora_Id_Transportadora
GROUP BY t.Nome_Transportadora
HAVING SUM(p.Valor_Pedido) > 0
ORDER BY Valor_Total_Carga DESC;

```

Algoritmo 24 – Consulta 22 (PostgreSQL)

nome_transportadora	valor_faturado	valor_entregue	valor_pendente	valor_total_carga
Pássaro Marron Logística	945.00	null	null	945.00
Correios Sedex	620.00	null	null	620.00
Trans Taubaté Expresso	null	null	null	453.00
Latam Cargo	null	null	299.00	299.00
Azul Cargo	null	298.00	null	298.00
TNT Mercurio	null	null	250.00	250.00
Total Express	null	null	250.00	250.00
Jadlog	null	235.00	null	235.00
Loggi	null	199.00	null	199.00
Braspress	null	null	122.50	122.50

Figura 27 – Consulta 22 (PostgreSQL)

- Consulta 23 -

```

SELECT
    c.Nome_Cliente,
    ARRAY_AGG(prod.Nome_Produto) AS Molhos_Comprados,
    MAX(pag.Valor_Pagamento) AS Maior_Pagamento_Registrado,
    CASE WHEN MAX(pag.Valor_Pagamento) > 500 THEN 'Cliente Ouro' ELSE
'Cliente Prata' END AS Classificacao
FROM Cliente c
JOIN Pedido p ON c.Id_Cliente = p.fk_Cliente_Id_Cliente
JOIN Item i ON p.Id_Pedido = i.fk_Pedido_Id_Pedido
JOIN Produto prod ON i.fk_Produto_Id_Produto = prod.Id_Produto
JOIN Pagamento pag ON p.Id_Pedido = pag.fk_Pedido_Id_Pedido
GROUP BY c.Nome_Cliente;

```

Algoritmo 25 – Consulta 23 (PostgreSQL)

nome_cliente	molhos_comprados	maior_pagamento_registrado	classificacao
Mundo Verde Taubaté	("Maionese com Alho")	199.00	Cliente Prata
Empório Fil Vale	("Molho Barbecue Zero")	453.00	Cliente Prata
Burger King Taubaté	("Maionese Tradicional Zero")	945.00	Cliente Ouro
Supermercado Shibata	("Tempero Frango Zero")	250.00	Cliente Prata
McDonalds	("Ketchup Zero Sódio")	620.00	Cliente Ouro
Distribuidora Saudável	("Cáida de Caramelo Zero")	122.50	Cliente Prata
Supermercado Semar	("Cáida de Chocolate Zero")	235.00	Cliente Prata
Nutri Store Suplementos	("Molho Shoyu Zero Sódio")	250.00	Cliente Prata
Armazém Natural	("Creme de Avelã Zero")	299.00	Cliente Prata
Atacadão S.A	("Molho Mostarda Zero")	298.00	Cliente Prata

Figura 28 – Consulta 23 (PostgreSQL)

- Consulta 24 -

```
SELECT
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Valor_Pedido) AS
Mediana_Faturamento,
    ROUND(AVG(Valor_Pedido), 2) AS Media_Tradicional,
    MIN(Valor_Pedido) AS Pior_Venda,
    MAX(Valor_Pedido) AS Melhor_Venda
FROM Pedido;
```

Algoritmo 26 – Consulta 24 (PostgreSQL)

mediana_faturamento	media_tradicional	pior_venda	melhor_venda
274	367.15	122.50	945.00

Figura 29 – Consulta 24 (PostgreSQL)

- Consulta 25 -

```
WITH Estatisticas AS (
    SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Valor_Pedido) AS
Mediana FROM Pedido
)
SELECT
    v.Nome_Vendedor,
    p.Valor_Pedido,
    e.Mediana
FROM Pedido p
CROSS JOIN Estatisticas e
JOIN Vendedor v ON p.fk_Vendedor_Id_Vendedor = v.Id_Vendedor
WHERE p.Valor_Pedido > e.Mediana
ORDER BY p.Valor_Pedido DESC;
```

Algoritmo 27 – Consulta 25 (PostgreSQL)

nome_vendedor	valor_pedido	mediana
Thiago Alvarenga	945.00	274
Marcelo Nogueira	620.00	274
Camila Razeide	453.00	274
Bruno Meireles	299.00	274
Diego Farias	298.00	274

Figura 30 – Consulta 25 (PostgreSQL)

- Consulta 26 -

```
SELECT
    Id_Cliente,
    UPPER(Nome_Cliente) AS Cliente_Capslock,
    COALESCE(NULLIF(Telefone_Cliente, ''), 'SEM TELEFONE') AS
Contato_Principal,
    SUBSTRING(Cpf_Cliente FROM 1 FOR 3) || '***' AS Documento_Mascarado
FROM Cliente
ORDER BY Nome_Cliente;
```

Algoritmo 28 – Consulta 26 (PostgreSQL)

id_cliente	cliente_capslock	contato_principal	documento_mascarado
9	ARMAZÉM NATURAL	(21) 2000-9999	999***
8	ATACADÃO S.A.	(11) 3000-8888	888***
3	BURGER KING TAUBATÉ	(12) 3632-3333	333***
6	DISTRIBUIDORA SAÚDEVEL	(11) 4002-6666	666***
5	EMPÁRIO FIT VALE	(12) 3621-5555	555***
4	MCDONALDS	(12) 3632-4444	444***
1	MUNDO VERDE TAUBATÉ	(12) 3632-1111	111***
7	NUTRI STORE SUPLEMENTOS	(11) 4002-7777	777***
2	SUPERMERCADO SEMAR	(12) 3622-2222	222***
10	SUPERMERCADO SHIBATA	(12) 3624-0000	000***

Figura 31 – Consulta 26 (PostgreSQL)

- Consulta 27 -

```
WITH Rank_Vendas AS (
    SELECT
        Id_Pedido,
        Status_Pedido,
        Valor_Pedido,
```

```

        ROW_NUMBER() OVER (PARTITION BY Status_Pedido ORDER BY
Valor_Pedido DESC) AS Ranking_Interno
        FROM Pedido
    )
    SELECT * FROM Rank_Vendas WHERE Ranking_Interno = 1;

```

Algoritmo 29 – Consulta 27 (PostgreSQL)

id_pedido	status_pedido	valor_pedido	ranking_interno
8	Entregue	298.00	1
1	Faturado	945.00	1
9	Pendente	299.00	1
3	Separacao	453.00	1

Figura 32 – Consulta 27 (PostgreSQL)

- Consulta 28 -

```

SELECT
    Data_Pedido,
    SUM(Valor_Pedido) AS Faturamento_Diario,
    ROUND(AVG(SUM(Valor_Pedido)) OVER (
        ORDER BY Data_Pedido
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2) AS Media_Movel_3_Dias
FROM Pedido
GROUP BY Data_Pedido
ORDER BY Data_Pedido;

```

Algoritmo 30 – Consulta 28 (PostgreSQL)

data_pedido	faturamento_diario	media_movel_3_dias
2026-05-24	298.00	298.00
2026-05-25	945.00	621.50
2026-05-26	620.00	621.00
2026-05-27	453.00	672.67
2026-05-28	199.00	424.00
2026-05-29	235.00	295.67
2026-05-30	372.50	266.83
2026-05-31	549.00	385.50

Figura 33 – Consulta 28 (PostgreSQL)

- Consulta 29 -

```

WITH Receita_Produtos AS (

```

```

SELECT
    prod.Nome_Produto,
    SUM(i.Quantidade_Item * i.Valor_Unitario - i.Desconto_Item) AS
Total_Receita
FROM Produto prod
JOIN Item i ON prod.Id_Produto = i.fk_Produto_Id_Produto
GROUP BY prod.Nome_Produto
),
Total_Geral AS (
    SELECT SUM(Total_Receita) AS Receita_Total_Empresa FROM
Receita_Produtos
),
Acumulado AS (
    SELECT
        rp.Nome_Produto,
        rp.Total_Receita,
        SUM(rp.Total_Receita) OVER (ORDER BY rp.Total_Receita DESC) AS
Receita_Acumulada,
        tg.Receita_Total_Empresa
    FROM Receita_Produtos rp
    CROSS JOIN Total_Geral tg
)
SELECT
    Nome_Produto,
    Total_Receita,
    ROUND((Receita_Acumulada / Receita_Total_Empresa) * 100, 2) AS
Porcentagem_Acumulada,
    CASE
        WHEN (Receita_Acumulada / Receita_Total_Empresa) <= 0.70 THEN
'Curva A (Alto Valor)'
        WHEN (Receita_Acumulada / Receita_Total_Empresa) <= 0.90 THEN
'Curva B (Médio Valor)'
        ELSE 'Curva C (Baixo Valor)'
    END AS Classificacao_ABC
FROM Acumulado
ORDER BY Total_Receita DESC;

```

Algoritmo 31 – Consulta 29 (PostgreSQL)

nome_produto	total_receita	porcentagem_acumulada	classificacao_abc
Maionesa Tradicional Zero	945.00	25.74	Curva A (Alto Valor)
Ketchup Zero Sódio	620.00	42.63	Curva A (Alto Valor)
Molho Barbecue Zero	453.00	54.96	Curva A (Alto Valor)
Creme de Avelã Zero	299.00	63.11	Curva A (Alto Valor)
Molho Mostarda Zero	298.00	71.22	Curva B (Médio Valor)
Molho Shoyu Zero Sódio	250.00	84.84	Curva B (Médio Valor)
Tempero Frango Zero	250.00	84.84	Curva B (Médio Valor)
Calda de Chocolate Zero	235.00	91.24	Curva C (Baixo Valor)
Maionesa com Alho	199.00	96.66	Curva C (Baixo Valor)
Calda de Caramelo Zero	122.50	100.00	Curva C (Baixo Valor)

Figura 34 – Consulta 29 (PostgreSQL)

- Consulta 30 -

```
SELECT
    p.Id_Pedido,
    p.Data_Pedido,
    p.Valor_Pedido,
    c.Nome_Cliente,
    JSONB_BUILD_OBJECT(
        'logistica', JSONB_BUILD_OBJECT(
            'transportadora', t.Nome_Transportadora,
            'status', r.Status_Entrega,
            'previsao', r.Data_Expedicao
        ),
        'financeiro', JSONB_BUILD_OBJECT(
            'metodo', pag.Metodo_Pagamento,
            'status_pagamento', pag.Status_Pagamento,
            'nota_fiscal', nf.Chave_Acesso
        ),
        'itens', (
            SELECT JSONB_AGG(JSONB_BUILD_OBJECT(
                'produto', prod.Nome_Produto,
                'qtd', i.Quantidade_Item,
                'subtotal', (i.Quantidade_Item * i.Valor_Unitario -
i.Desconto_Item)
            ))
            FROM Item i
            JOIN Produto prod ON i.fk_Produto_Id_Produto =
prod.Id_Produto
            WHERE i.fk_Pedido_Id_Pedido = p.Id_Pedido
        )
    ) AS Payload_API_Completo
FROM Pedido p
JOIN Cliente c ON p.fk_Cliente_Id_Cliente = c.Id_Cliente
JOIN Transportadora t ON p.fk_Transportadora_Id_Transportadora =
t.Id_Transportadora
JOIN Romaneio r ON p.fk_Romaneio_Id_Romaneio = r.Id_Romaneio
JOIN Pagamento pag ON p.Id_Pedido = pag.fk_Pedido_Id_Pedido
JOIN Nota_Fiscal nf ON p.Id_Pedido = nf.fk_Pedido_Id_Pedido;
```

Algoritmo 32 – Consulta 30 (PostgreSQL)

id_pedido	data_pedido	valor_pedido	nome_cliente	payload_api_completo
1	2026-05-25	945.00	Burger King Taubaté	[{"financeiro": {"metodo": "Pix", "nota_fiscal": "35260511111110001155001000000011234567890", "status_pagamento": "Aprovado"}, "itens": [{"produto": "Maionese Tradicional Zero", "qtd": 50, "subtotal": 945.0}], "logistica": {"previsao": "2026-05-30", "status": "Em Rota", "transportadora": "P&Sarrá Marron Logística"}}]
2	2026-05-26	620.00	McDonalds	[{"financeiro": {"metodo": "Boleto", "nota_fiscal": "3526052222222200012255001000000021234567890", "status_pagamento": "Aguardando"}, "itens": [{"produto": "Ketchup Zero Sódio", "qtd": 40, "subtotal": 620.0}], "logistica": {"previsao": "2026-05-31", "status": "Pendente", "transportadora": "TransTaubaté Expresso"}}]
3	2026-05-27	453.00	Empório Fil Vale	[{"financeiro": {"metodo": "Cartão de Crédito", "nota_fiscal": "3526053333333000133550010000000031234567890", "status_pagamento": "Aprovado"}, "itens": [{"produto": "Molho Barbecue Zero", "qtd": 20, "subtotal": 453.0}], "logistica": {"previsao": "2026-06-01", "status": "Pendente", "transportadora": "TransTaubaté Expresso"}}]
4	2026-05-28	199.00	Mundo Verde Taubaté	[{"financeiro": {"metodo": "Pix", "nota_fiscal": "3526054444444000144550010000000041234567890", "status_pagamento": "Aprovado"}, "itens": [{"produto": "Maionese com Alho", "qtd": 10, "subtotal": 199.0}], "logistica": {"previsao": "2026-05-28", "status": "Entregue", "transportadora": "Loggi"}}]
5	2026-05-29	235.00	Supermercado Semar	[{"financeiro": {"metodo": "Cartão de Débito", "nota_fiscal": "3526055555555000155550010000000051234567890", "status_pagamento": "Aprovado"}, "itens": [{"produto": "Caldão de Chocolate Zero", "qtd": 10, "subtotal": 235.0}], "logistica": {"previsao": "2026-05-29", "status": "Entregue", "transportadora": "Jadlog"}}]
6	2026-05-30	122.50	Distribuidora Saudável	[{"financeiro": {"metodo": "Pix", "nota_fiscal": "3526056666666000166550010000000061234567890", "status_pagamento": "Recusado"}, "itens": [{"produto": "Caldão de Caramelo Zero", "qtd": 5, "subtotal": 122.5}], "logistica": {"previsao": "2026-06-02", "status": "Pendente", "transportadora": "Binapress"}}]
7	2026-05-30	250.00	Nutri Store Suplementos	[{"financeiro": {"metodo": "Pix", "nota_fiscal": "3526057777777000177550010000000071234567890", "status_pagamento": "Aprovado"}, "itens": [{"produto": "Molho Shoyu Zero Sódio", "qtd": 15, "subtotal": 250.0}], "logistica": {"previsao": "2026-06-03", "status": "Pendente", "transportadora": "TNT Mercuro"}}]
8	2026-05-24	298.00	Atacadão S.A.	[{"financeiro": {"metodo": "Cartão de Crédito", "nota_fiscal": "3526058888888000188550010000000081234567890", "status_pagamento": "Aprovado"}, "itens": [{"produto": "Molho Mostarda Zero", "qtd": 20, "subtotal": 298.0}], "logistica": {"previsao": "2026-05-27", "status": "Entregue", "transportadora": "Azul Cargo"}}]
9	2026-05-31	299.00	Armazém Natural	[{"financeiro": {"metodo": "Cartão de Débito", "nota_fiscal": "3526059999999000199550010000000091234567890", "status_pagamento": "Aprovado"}, "itens": [{"produto": "Creme de Avelã Zero", "qtd": 10, "subtotal": 299.0}], "logistica": {"previsao": "2026-06-04", "status": "Pendente", "transportadora": "Latam Cargo"}}]
10	2026-05-31	250.00	Supermercado Shibata	[{"financeiro": {"metodo": "Pix", "nota_fiscal": "35260600000000000100550010000000101234567890", "status_pagamento": "Aguardando"}, "itens": [{"produto": "Tempero Frango Zero", "qtd": 20, "subtotal": 250.0}], "logistica": {"previsao": "2026-06-05", "status": "Pendente", "transportadora": "Total Express"}}]

Figura 35 – Consulta 30 (PostgreSQL)

4 CONCLUSÃO

O desenvolvimento deste projeto consolidou a aplicação prática dos conceitos de modelagem e administração de sistemas de bancos de dados para a criação de um Sistema de Vendas Alimentícias robusto. O objetivo de planejar, modelar e construir um banco de dados relacional que gerencie com segurança todo o processo de vendas (desde o pedido até a logística e faturamento) foi plenamente atingido, utilizando o BrModelo para as fases conceitual e lógica. A implementação física foi realizada no ambiente PostgreSQL (conforme a metodologia) com uma arquitetura de nove tabelas interligadas, que representam o ciclo completo de vendas: Item, Pedido, Vendedor, Cliente, Romaneio, Transportadora, Pagamento, Nota Fiscal e Produto.

A estruturação seguiu rigorosamente os princípios do modelo relacional e técnicas de normalização de Rob e Coronel, garantindo a integridade dos dados e o alinhamento com as regras de negócio definidas. O projeto foi validado com a execução das trinta consultas SQL avançadas, baseadas nas técnicas de Thomas Nield, que comprovaram a capacidade do sistema em gerar relatórios gerenciais complexos e analíticos. Em relação ao cronograma de atividades, todas as etapas, desde o levantamento bibliográfico até a homologação das consultas, foram cumpridas com sucesso.

Como sugestões para a expansão futura do projeto, seria pertinente desenvolver um módulo de controle de inventário e estoque mais detalhado, que permita a baixa automática dos produtos a cada venda registrada, e a criação de um painel de BI integrado para visualização dos relatórios gerenciais complexos gerados pelas consultas SQL.

REFERÊNCIAS

NIELD, Thomas. **Introdução à linguagem SQL**. São Paulo: Novatec Editora, 2016.

ROB, Peter; CORONEL, Carlos. **Sistemas de banco de dados**: projeto, implementação e administração. São Paulo: Cengage Learning, 2011.